Appendix A

Tutorial

This tutorial will teach you how to build an efficient ray tracer that runs on dedicated hardware using a high-level ray tracing framework. Before starting this tutorial, you should be comfortable with:

- Basic C++ programming skills
  - Variables, types, generics, functions, classes/structs, pointers/references
  - C++ standard library (e.g. std::vector)
  - Knowledge of more complex C++ features is **not** required, barrier of entry for non-experienced C++ developers is thus quite low
- Basic linear algebra
  - Vectors and matrices
  - Dot and cross products
- Some computer graphics knowledge
  - Experience with GPU shaders
  - Ray tracing fundamentals
  - Triangle representation (vertices and indices)
  - Surface normals and normal transformations
  - UV coordinates and texture mapping
  - Barycentric coordinates
  - Simple lighting models such as Phong

In this tutorial you'll build a capable and efficient ray tracer with seven incremental projects:

1. Hello Triangle
2. Hello Sphere
3. Camera
4. Diffuse Lighting
5. Anti-Aliasing
6. Reflection
7. Passing Vertex Data

A.0 Development environment

This section contains instructions for setting up a basic development environment used to run Vulkan applications. All tools used in this project are compatible with Windows, Linux and macOS. Installation steps may differ between platforms.

This project requires ray tracing capable hardware. If you have any of these GPUs you can run the project at full efficiency using hardware acceleration:

- NVIDIA GeForce RTX 20 series and newer [15]
- AMD Radeon RX 6000 series and newer [16]
- Intel Arc Desktop and Professional [17]

Older GPUs may have a slower, software-based fallback option for ray tracing.

If your machine does not support GPU-based ray tracing, a fallback option like LLVMpipe can be used instead. This driver will execute Vulkan applications on the CPU instead of the GPU, resulting in a significant performance penalty.

A.0.1 Vulkan SDK

The Vulkan SDK is the most important dependency. It includes the required header files, validation layers and Vulkan loader functions. These functions are used to dynamically load the Vulkan driver functions at runtime. Detailed installation instructions for all three supported platforms can be found on the LunarG website.
You can ensure correct installation by executing the *vkcube* demo application included with the SDK.

A.0.2 Slang

This tutorial uses the Slang shading language. Since Vulkan only supports the SPIR-V representation, any shading language that supports conversion to SPIR-V is supported. This tutorial focuses on the aforementioned Slang. This language is syntactically very close to C and HLSL with many convenient features other shading languages lack, such as operator overloading and generics. Another useful feature, specifically for Vulkan applications, is automatic resource binding declarations. This feature will be further expanded on in the Hello Triangle tutorial.

The Slang compiler *slangc* is provided by the Vulkan SDK, you can ensure proper installation using this command:

```
$ slangc -v
```

The provided CMake project template will automatically compile all **.slang* files inside the *shaders/* folder on project build. The default target directory is *build/shaders/*.

A.0.3 C++ Development

This project requires a C++ compiler that supports C++20 and the CMake build tool. You are free to pick any IDE / build system you want (e.g. Visual Studio, CLion, Xcode, Make, Ninja, ...).

GLFW

The tutorial uses the GLFW library for creating windows. On Windows and macOS this should work out-of-the-box. Linux requires additional installation of additional dependencies. Installation instructions for Linux can be found on the GLFW website.

Building

The following commands demonstrate how to build the project - including all dependencies - from scratch using the Ninja build system. Use the appropriate commands for your OS of choice:

```
$ mkdir build
$ cd build
$ cmake -G Ninja ..
$ ninja
```

Change the *-G* argument to use a build system other than Ninja. To create an optimized release build, use this command instead:

```
...
$ cmake -G Ninja -DCMAKE_BUILD_TYPE=Release ..
...
```

After running the above commands for the first time, for future builds you can omit the CMake command and just compile using your build system (in this example Ninja). If you add or delete new files, you need to update the appropriate *CMakeLists.txt* file. For example if you add another file inside the *raytracer/src/* folder, the *raytracer/CMakeLists.txt* file needs to be updated with the new file name. The same applies for files inside the *shaders/* and *engine/* folders.

You are now ready to run the *raytracer/src/main.cpp* file:

```
$ ./raytracer/raytracer
```

Each tutorial generates a separate executable. These executables are also available inside the *raytracer/* folder. Example on how to run the first part tutorial ”Hello Triangle”:

```
$ ./raytracer/01_hello_triangle
```

Now that the development environment is done, we are ready to start with the main tutorial. You can write along with the tutorial in the empty file *raytracer/src/main.cpp*.

A.1 Hello Triangle

```
- raytracer/src/tutorial/01_hello_triangle.cpp
- shaders/tutorial/01_hello_triangle.slang
```

In this first tutorial, the building blocks for all future tutorials will be laid out. This includes the following steps:

- Window Creation
- Ray Tracing Framework Setup
- Writing a Simple Ray Tracing Shader
- Shader Resources
- Shader Stages
- Rendering a Frame to the Screen

A.1.1 Window Creation

We'll start with the window creation. Creating a window is not part of the abstraction layer, this gives the developer the freedom to use any window library with Vulkan support they want. In this tutorial we'll use the GLFW library. A small abstraction over GLFW is available in the *raytracer/src/util/* folder. Inner workings of this abstraction are not detailed in this tutorial. But due to its small size, it should be easily understandable and expandable.

Create a new window:

```cpp
#include <engine/engine.hpp>
#include "util/window.hpp"
#include <glm/glm.hpp>

const int WIDTH = 1500;
const int HEIGHT = 1000;
const std::string NAME = "01 - Hello Triangle";

int main() {
    Window window(WindowInfo{
        .width = WIDTH,
        .height = HEIGHT,
        .name = NAME
    });
}
```

Code 4: Hello Triangle - Window Setup

We include the ray tracing engine called `engine`, specifically the `engine.hpp` header, which includes all other necessary headers we will soon use. We also need to include the previously mentioned `Window` class from the util folder. In this tutorial, the GLM library is used for various useful vector math operations.

Most information in this abstraction layer is passed through structs instead of function parameters. The C++20 feature *Designated Initializers* exposes the parameter names and makes the code more readable.
In the Hello Triangle - Window Setup code block above we set the `width`, `height` and `name` of the window. Because all values will be used again in the future, they are already moved outside the function as constants.

A.1.2 Ray Tracing Framework Setup

Next we need to set up connection to a Vulkan capable device. To do that we need a `engine::Context` and a `engine::Device` inside our main function.

```
// ...
const engine::Context context(engine::ContextInfo{
    .appName = NAME,
    .enableValidation = true,
    .requiredExtensions = window.getRequiredExtensions()
});

const Dimensions dimensions(WIDTH, HEIGHT);
engine::Device device(context, engine::RenderTarget{
    .surface = window.createSurface(context),
    .dimensions = dimensions,
    .vsync = true
});
// ...
```

Code 5: Hello Triangle - Device Setup

The *enableValidation* parameter enables the Vulkan validation layers. These layers will check every interaction with Vulkan for errors and report them via the command line. We enable them here, so potential issues are reported to us. It is recommended to turn off validation for release builds, as they are quite heavy on the CPU [18]. This can be done by checking if the **DEBUG** macro is defined or not.
The *requiredExtensions* contains a list of extensions that need to be supported by a Vulkan implementation. This list is provided by the GLFW library.

In order to present an image to the screen, the device needs to know the dimensions of the window. This value is then passed to the `engine::Device`. The device also needs access to the surface it will render on. Just like the `requiredExtensions`, this surface is also provided by the GLFW library.
The last parameter `vsync` refers to *vertical sync*. This means that frames will be produced at the same rate as the screen refreshes. Meaning for a 60Hz monitor, 60 frames are produced every second. Setting this value to `false` will remove this restriction and run the ray tracer at full speed, resulting in a close to 100% GPU usage.

Running the code now will output the name of the physical device that will be used for ray tracing.

A.1.3 Writing a Simple Ray Tracing Shader

Now that the device is set up, we can start preparing the shader that will be used by the ray tracing process. First create a new shader **shaders/main.slang**. Remember to add the new shader to the *shaders/CMakeLists.txt* file.

During ray tracing, the engine needs to know three things:

1. Where do rays originate?
2. What happens if a ray hits geometry?
3. What happens if a ray misses geometry?

We'll need a shader function to handle each question, starting with the ray origin.

```
static const float cameraDepth = -1.0;

[shader("raygeneration")]
void raygenMain() {
    int2 launchId = DispatchRaysIndex().xy;
    int2 launchSize = DispatchRaysDimensions().xy;

    float2 coords = float2(launchId) / float2(launchSize) * 2.0 - 1.0;
    coords.x *= float(launchSize.x) / float(launchSize.y);
    const float3 origin = float3(coords, cameraDepth);

    RayDesc ray;
    ray.Origin = origin;
    ray.Direction = float3(0.0, 0.0, 1.0);
    ray.TMin = 0.0;
    ray.TMax = 1e4;
}
```

Code 6: Hello Triangle - Ray Description

Create a new function with a fitting name, `raygenMain` in this case. Add the attribute `[shader("raygeneration")]` on top of the function, so the Slang compiler knows what is will be used for. This function is called once per pixel. We can get the coordinates of the current pixel by calling the `DispatchRaysIndex()` function. The origin point (0, 0) is located in the top left corner. `DispatchRaysDimensions()` returns the size of the frame, we previously set this to 1500x1000.
We can now use these two values to calculate the origin of the a new ray.

Lastly, we create a new ray with the built-in type `RayDesc`. We use simple parallel projection in this example, so the ray direction is just set to *1.0* (forward). Set the minimum distance to the closest geometry to *0.0* and clip everything beyond *10'000*.

Now we need to trace that ray against geometry. To do that, we need access to our geometry through an acceleration structure (we will create said acceleration structure with actual data a bit later). To output a color to the screen, write access to an image (texture) is also required.

```
RaytracingAccelerationStructure accelerationStructure;
RWTexture2D<float4> outImage;

struct HitPayload {
    float3 color;
}

static const float cameraDepth = -1.0;

[shader("raygeneration")]
void raygenMain() {
    // ...

    HitPayload payload;
    payload.color = float3(0.0);

    TraceRay(accelerationStructure, RAY_FLAG_NONE, 0xFF, 0, 1, 0, ray,
     ↪  payload);
    outImage[launchId] = float4(payload.color, 1.0);
}
```

Code 7: Hello Triangle - Tracing a Ray

Define the two resources at the top of the file. To trace a ray, call the built-in function `TraceRay`. The most important parameters are the acceleration structure, the ray to trace and the payload. The payload can contain any data you like, for now the color is enough. The other parameters to `TraceRay` will be explained in future chapters. The last step is to set the color from the payload to the current pixel on a writable texture.
A ray is now generated for each pixel on the frame and traced against an acceleration structure.

But what happens if a ray hits or misses geometry? Two more entry points `closesthit` and `miss` are needed:

```
// ...

[shader("closesthit")]
void closestHitMain(inout HitPayload payload, in
 ↪  BuiltInTriangleIntersectionAttributes attr) {
    payload.color = float3(1.0 - attr.barycentrics.x - attr.barycentrics.y,
     ↪  attr.barycentrics.x, attr.barycentrics.y);
}

[shader("miss")]
void missMain(inout HitPayload payload) {
    payload.color = float3(0.75, 0.8, 1.0);
}
```

Code 8: Hello Triangle - Closest Hit and Miss

For each ray, the *closesthit* marked function is called once. For now set the color of the hit point to the barycentric coordinates of that point. The barycentric coordinates are provided via the `BuiltInTriangleIntersectionAttributes` struct.
If no intersection is found (*miss* shader), set the payload color to a light blue.

That's the entire shader needed for the Hello Triangle tutorial.

A.1.4 Shader Resources

We now jump back to the C++ code. In the previous chapter, we defined in the shader that access to an acceleration structure and a texture is needed. The acceleration structure needs triangle data. For the first tutorial this data is self-defined before the main function:

```
struct Vertex {
    glm::vec4 position;
};

const std::vector<Vertex> vertices = {
    {{ 0.0f, 0.75f, 0.0f, 0.0f }},
    {{ -0.75f, -0.75f, 0.0f, 0.0f }},
    {{ 0.75, -0.75f, 0.0f, 0.0f }}
};
const std::vector<uint32_t> indices = { 0, 1, 2 };
```

Code 9: Hello Triangle - Vertex Data

For reasons outside the scope of this tutorial, data on the GPU needs to be aligned to 16 bytes. Although only three floats are needed to represent a vertex in 3D space, an additional float is required to extend the data to a total of 16 bytes.

Now we need to send this data to the GPU (acceleration structure building takes place on the GPU). To do that a structured buffer is created. A structured buffer is simply an array of custom data on the GPU, this buffer can be arbitrarily large (millions of triangles in large scenes). Create two small structured buffers, one for the vertices and one for the indices:

```
// ...

auto vertexBuffer = device.structuredBuffer<Vertex>(vertices.size(), {
↪  .accelerationStructureInput = true });
vertexBuffer.fill(vertices.data());
auto indexBuffer = device.structuredBuffer<uint32_t>(indices.size(), {
↪  .accelerationStructureInput = true });
indexBuffer.fill(indices.data());

// ...
```

Code 10: Hello Triangle - Structured Buffers

In addition to the size of the structured buffer, also set the flag `accelerationStructureInput` to `true`. This ensures correct acceleration structure data handling by the framework.

To create an actual acceleration structure, use the provided `AccelerationStructureBuilder` and call `add` with the vertex and index buffers:

```
// ...

engine::AccelerationStructureBuilder asBuilder;
asBuilder.add(vertexBuffer, indexBuffer);
auto accelerationStructure = device.accelerationStructure(asBuilder);

// ...
```

Code 11: Hello Triangle - Acceleration Structure

That's everything needed to create an acceleration structure, on to the second resource needed by the shader, the output texture. This one is very simple:

```
// ...

auto outputTexture = device.texture(engine::TextureInfo{ .dimensions =
↪  dimensions });

// ...
```

Code 12: Hello Triangle - Output Texture

Just request a texture with the same dimensions as the window. You can find a full list of all available shader resources in chapter 4.2.1. Now let's expose these two resources to a shader with the `ShaderResources` class:

```
// ...

engine::ShaderResources shaderResources;
shaderResources
    .expose(accelerationStructure, engine::ShaderStageFlags::Raygen)
    .expose(outputTexture, engine::ShaderStageFlags::Raygen);

// ...
```

Code 13: Hello Triangle - Shader resources

The order of resources must match with the order defined in the shader, otherwise the Slang automatic binding feature will not work.
Right now, the resources are only needed in the *Raygen* stage. Resources can also be exposed to multiple stages at the same time, by combining flags with the *or* operator.

A.1.5 Shader Stages

Before a frame can be rendered, the framework needs to know where our shaders are and the stages within them:

```
// ...

engine::Shader shader("shaders/main.spv");
engine::ShaderStages shaderStages;
shaderStages
    .addRaygen(engine::ShaderStage(shader, "raygenMain"))
    .addGroup(engine::TriangleHitGroup(engine::ShaderStage(shader,
    ↪  "closestHitMain")))
    .addMiss(engine::ShaderStage(shader, "missMain"));

// ...
```

Code 14: Hello Triangle - Shader Stages

Add the three shader stages (raygen, closestHit and miss) defined in the shader to the stages. The second parameter (`entryPoint`) must match the corresponding function name inside the shader file.
The *main.slang* file is automatically compiled into the *main.spv* file on project build (remember that only SPIR-V shaders are supported).

A.1.6 Rendering a Frame to the Screen

The last step before something is finally shown on the screen, is the main render loop. With the use of the framework this is again very simple:

```
// ...

engine::Renderer renderer(device, shaderResources, shaderStages);

while (!window.shouldClose()) {
    window.pollEvents();
    renderer.render(device, outputTexture);
}

device.wait();
```

Code 15: Hello Triangle - Frame Render

Create a `engine::Renderer` with the `device`, `shaderResources` and `shaderStages`. Then create a while-loop that runs until the window is closed. Don't forget to poll the window events inside the loop, else any interaction with the window (including closing) won't be handled. To render a frame, simply call the `render` method with the texture you want to see on the screen.

Before the window is closed at the end of the main function, you should also call `device.wait()`. This ensures the GPU is idle before all resources are destroyed and the application terminates.

Running the application now should finally present an image:

Figure A.1: Hello Triangle - Final result

You may notice that the colors are brighter than expected. This is because by default, the framework applies a conversion from linear color space to sRGB. The effects of this will be much more noticeable later on in the tutorial. If you wish to disable the color space conversion, set the `srgb` field inside the `engine::RenderTarget` struct passed to the `engine::Device` constructor to `false`. Disabling this option is not recommended because modern displays expect non-linear colors like sRGB [7]. More information from this topic is available on chapter 4.1.5.

A.2 Hello Sphere

```
- raytracer/src/tutorial/02_hello_sphere.cpp
- shaders/tutorial/02_hello_sphere.slang
```

Now that the groundwork is laid out and the first triangle is visible on the screen, rendering a sphere is the next target. More accurately a mathematically defined sphere, no triangle composition. This is going to be set up quite differently than triangles. Let's start with the shader this time (*main.slang*).

A.2.1 Sphere Intersection

```
RaytracingAccelerationStructure accelerationStructure;

struct Sphere {
    float3 center;
    float radius;
}
StructuredBuffer<Sphere> spheres;

RWTexture2D<float4> outImage;

// ...
```

Code 16: Hello Sphere - Sphere Definition and Creation

We define a sphere with a center point and a radius. In the last chapter, a structured buffer of vertex and index data was created for building an acceleration structure. But access to the vertex data was not necessary. This time access is required for the sphere intersection calculation. So this information is also set as a shader resource.
Next up is the logic to determine, if a ray intersects a sphere:

```
// Ray-sphere intersection
// By Inigo Quilez, from https://iquilezles.org/articles/spherefunctions/
float sphIntersect(const Sphere s, float3 ro, float3 rd) {
    float3 oc = ro - s.center;
    float b = dot(oc, rd);
    float c = dot(oc, oc) - s.radius * s.radius;
    float h = b * b - c;
    if (h < 0.0) {
        return -1.0;
    }
    h = sqrt(h);
    return -b - h;
}
```

Code 17: Hello Sphere - Sphere Intersection

This function is is an adapted version from a collection of ray-sphere intersection tests by Inigo Quilez (https://iquilezles.org/articles/spherefunctions/) . It returns the distance between the ray origin and the closest point on the sphere.
This alone is not quite enough for the intersection test. Just like in the *closestHit* function of a triangle, an entry point is required. An intersection-type entry point looks as follows:

```
[shader("intersection")]
void sphereIntersect() {
    const Sphere sphere = spheres[InstanceID()];
    float hit = sphIntersect(sphere, WorldRayOrigin(), WorldRayDirection());

    if (hit > 0.0) {
        ReportHit(hit, 0, 0);
    }
}
```

Code 18: Hello Sphere - Intersection Entry Point

This code snippet uses various built-in functions. The `WorldRayOrigin()` and `WorldRayDirection()` are fairly obvious. These just return the ray origin and ray direction from the calling `TraceRay(...)` function, in this case from the *raygeneration* entry point.
The `ReportHit(hit, 0, 0)` call is also quite obvious. If a hit is found, return the distance to said hit in the first parameter. The second parameter specifies the kind of hit (not used in this tutorial). The third parameter can contain additional intersection data that is passed to the *closestHit* function as the second parameter (like `BuiltInTriangleIntersectionAttributes` for a triangle closest hit).

The built-in `InstanceID()` function returns a user defined value, we will set this value in the C++ code.

A.2.2 Sphere Closest Hit

The *closestHit* function for procedural geometry is the same as for the triangle one. Since no attribute is passed from the *intersection* to the *closestHit* shader, the second parameter is (attributes) omitted.

```
[shader("closesthit")]
void sphereClosest(inout HitPayload payload) {
    payload.color = float3(1.0, 0.05, 0.05);
}
```

Code 19: Hello Sphere - Closest Hit Entry Point

For simplicity, let's just color the sphere a nice red.

A.2.3 Sphere on the C++ Side

The definition of the sphere is the same on the C++ side is the same as in the shader. Let's add two spheres to the scene, to the left and right of the triangle:

```
struct Sphere {
    glm::vec3 center;
    float radius;
};

int main() {
    // Vertex / index buffer definition.

    // Spheres:
    std::array<Sphere, 2> spheres = {
        Sphere{ glm::vec3(1.0, 0.0, 0.0), 0.3, },
        Sphere{ glm::vec3(-1.0, 0.0, 0.0), 0.3, },
    };
    auto sphereBuffer = device.structuredBuffer<Sphere>(spheres.size());
    sphereBuffer.fill(spheres.data());

    // ...
}
```

Code 20: Hello Sphere - C++ Sphere Definition

Copy the two spheres into another structured buffer. This is the buffer that is exposed in the shader.
As an optimization, each procedural shape needs to be wrapped inside an AABB. This makes sure, the possibly computational heavy intersection shader is only executed if the ray first hits the AABB. The framework contains a AABB struct with a minimum vector and maximum vector, representing the minimum and maximum corners of the box respectively, more information on AABBs can be found in this MSDN article.

Let's add the creation of the AABB to the `Sphere` struct:

```
// ...

struct Sphere {
    glm::vec3 center;
    float radius;

    engine::AABB aabb() const {
        return engine::AABB{ glm::vec3(-radius), glm::vec3(radius) };
    }
};

// ...
```

Code 21: Hello Sphere - Sphere AABB

Before adding the spheres to the acceleration structure, let's extend the shader resources and shader stages:

```
// ...

engine::ShaderResources shaderResources;
shaderResources
    .expose(accelerationStructure, engine::ShaderStageFlags::Raygen)
    .expose(sphereBuffer, engine::ShaderStageFlags::Intersection) // NEW
    .expose(outputTexture, engine::ShaderStageFlags::Raygen);

engine::Shader shader("shaders/main.spv");
engine::ShaderStages shaderStages;
shaderStages
    .addRaygen(engine::ShaderStage(shader, "raygenMain"))
    .addGroup(engine::TriangleHitGroup(engine::ShaderStage(shader,
    ↪  "closestHitMain")))
    .addGroup(engine::ProceduralHitGroup(
        engine::ShaderStage(shader, "sphereIntersect"),
        engine::ShaderStage(shader, "sphereClosest"))
    ) // NEW
    .addMiss(engine::ShaderStage(shader, "missMain"));

// ...
```

Code 22: Hello Sphere - Sphere Shader Resources and Stages

Expose the sphere buffer to the shader, alongside the acceleration structure and output texture. Additionally add a `engine::ProceduralHitGroup` to the shader stages with the correct *intersection* and *closestHit* entry points.

Let's finally add the two spheres to the acceleration structure:

```
// ...

engine::AccelerationStructureBuilder asBuilder;
asBuilder.add(vertexBuffer, indexBuffer, { .hitGroupIndex = 0 });
for (uint32_t i = 0; i < spheres.size(); ++i)
    asBuilder.add(spheres[i].aabb(), { .instanceId = i, .hitGroupIndex = 1
    ↪  });
auto accelerationStructure = device.accelerationStructure(asBuilder);

// ...
```

Code 23: Hello Sphere - Add Sphere to Acceleration Structure

The important part here is the parameter `hitShaderGroupIndex`. This is an index into the groups set in the `ShaderStages`. In this case **index 0** is the triangle hit group and **index 1** is the procedural hit group, because that's the order the stages were added in here: Hello Sphere - Sphere Shader Resources and Stages.
Because the `InstanceID` is accessed inside the shader to get the current sphere, the value is set to the sphere index.

Now let's look at the the current render result:

Figure A.2: Hello Sphere - Wrong First Result

The image is exactly the same, unfortunately. The reason being, the AABB is placed at the origin of the scene (0, 0, 0), but the spheres are located to the left and right of the origin, outside the AABB. One way to fix this is just to increase the AABB size. You can try this be using a custom value inside the `aabb()` function that is large enough to fit the spheres. But the problem is obvious, the AABBs are larger than the inner sphere, so the intersection is executed unnecessarily often.
A much better fix is moving the AABB to the sphere origin. This transformation is supported by the framework for both triangle and procedural geometry. The transformation is represented by a 4x3 matrix. Let's expand the `Sphere` struct again with a function that returns the sphere origin as a matrix:

```
struct Sphere {
    glm::vec3 center;
    float radius;

    engine::AABB aabb() const {
        return engine::AABB{ glm::vec3(-radius), glm::vec3(radius) };
    }

    glm::mat4x3 mat() const {
        glm::mat4x3 transform(1.0f);
        transform[3] = center;
        return transform;
    }
};
```

Code 24: Hello Sphere - Sphere Transformation Matrix

Since the `glm::mat4x3` is column-major, setting the fourth column (index 3) to the sphere origin vector yields the correct transformation matrix. This not only works with translation, but also scaling and rotation. This is very powerful, because we could implement something similar to Geometry Instancing. Rendering a complex mesh (with possibly millions of triangles) many times at different scale/rotation/translation is possible, without much overhead.

Anyway, the last thing to do now is apply the transformation to the AABB:

```
// ...

for (uint32_t i = 0; i < spheres.size(); ++i)
    asBuilder.add(spheres[i].aabb(), { .instanceId = i, .transform =
    ↪ spheres[i].mat(), .hitGroupIndex = 1 });

// ...
```

Code 25: Hello Sphere - Sphere AABB Transformed

Running the ray tracer again will finally show the spheres:

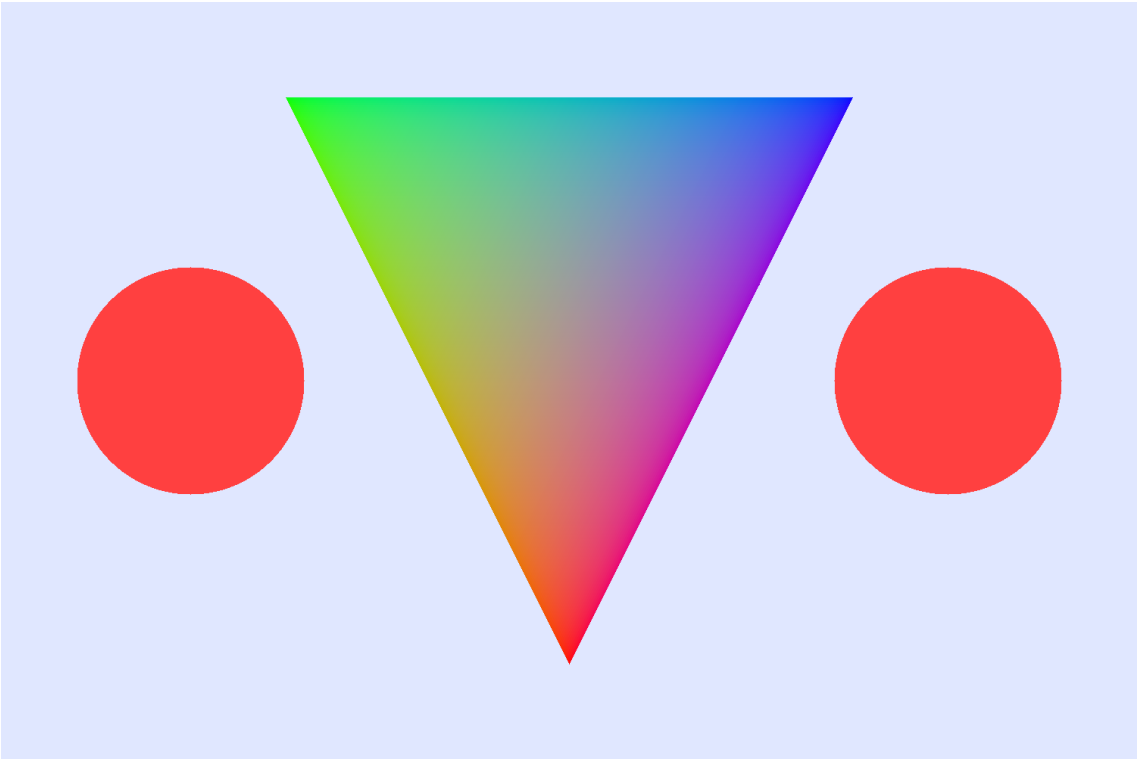

Figure A.3: Hello Sphere - Final result

The spheres look 2D right now, this can be fixed by applying something like depth cueing (darken the color the further away the object is from the camera). Try to implement this yourself. Tip: You can get the current ray distance with the built-in shader function `RayTCurrent()`.

A.3 Camera

```
- raytracer/src/tutorial/03_camera.cpp
- shaders/tutorial/03_camera.slang
```

Right now, rays are shot in parallel straight forward into the scene, originating from the 3D viewport. To improve upcoming renders, this chapter adds a perspective camera. This camera can be "controlled" from the C++ code, meaning mouse movement and other use interactions are possible (this is not directly covered in this tutorial). This chapter also introduces a new buffer type called **constant buffer**, this buffer is used to send frequently changing, non-array data to the GPU.
Let's start by defining the camera properties on the C++ side (*main.cpp*):

```
struct Camera {
    float meterPerPixel;
    float focalLength;
    glm::vec4 origin;
    glm::mat4 rotation;
};
```

Code 26: Camera - C++ Definition

There are many ways to achieve perspective projection. We use these four values:

- `meterPerPixel`: From this point on, all primitives are defined in meters. This value is needed for the conversion from device coordinates to meters.
- `focalLength`: This is the distance between the camera origin and the screen or viewport. A smaller value means the rays spread out more, widening the angle (more objects in the scene become visible).
- `origin`: The origin of the camera.
- `rotation`: Camera rotation as a 4x4 matrix.

Now let's create a constant buffer with the camera settings:

```
Camera camera{
    .meterPerPixel = 1.0 / static_cast<float>(WIDTH),
    .focalLength = 0.6,
    .origin = glm::vec4(0.0, 0.0, -2.0, 0.0),
    .rotation = glm::mat4(1.0)
};
auto cameraBuffer = device.constantBuffer<Camera>();
cameraBuffer.fill(&camera);

engine::ShaderResources shaderResources;
shaderResources
    .expose(accelerationStructure, engine::ShaderStageFlags::Raygen)
    .expose(cameraBuffer, engine::ShaderStageFlags::Raygen)
    // ...
```

Code 27: Camera - Camera Constant Buffer

Creation of the `ConstantBuffer` is similar to `StructuredBuffer` used before, with the exception of the missing `size` parameter, because constant buffers are not arrays.
The `focalLength` of 0.6 is an arbitrary, feel free to play around with this value. Additionally, move the camera little bit more back, so the sphere and triangle fit on the screen.

Let's move to the *main.slang* shader file and implement perspective. Start by adding the same `Camera` struct and referencing it in the `ConstantBuffer<Camera>` resource:

```
struct Camera {
    float meterPerPixel;
    float focalLength;
    float4 origin;
    float4x4 rotation;
};
ConstantBuffer<Camera, ScalarDataLayout> camera;
```

Code 28: Camera - Camera Shader Resource

As previously mentioned, data needs to be aligned to 16 bytes. This not only applies to entire structs, it also applies to array-type elements inside structs. Vectors and matrices are also treated as array types. In the `Camera` struct above, the `origin` needs to start at 16 bytes, but it actually starts at byte 8. This can be fixed in two ways. One possibility is to insert two 4-byte elements before the `origin` member **in the C++ code only**. The other, better way is to select a different memory layout inside the shader. This is the approach used in the code block Camera - Camera Shader Resource above. The memory layout used is called `ScalarDataLayout`. This layout is very close to the way C aligns data, so no padding is needed and the struct can be defined the same way in the shader and C++ code. More details on memory layout can be found in the official Vulkan documentation.

Now use the camera properties inside the `raygenMain` function:

```
[shader("raygeneration")]
void raygenMain() {
    // ...
    const float cx = (float(launchId.x) - float(launchSize.x) * 0.5) *
     ↪  camera.meterPerPixel;
    const float cy = (float(launchSize.y) * 0.5 - float(launchId.y)) *
     ↪  camera.meterPerPixel;

    RayDesc ray;
    ray.Origin = camera.origin.xyz;
    ray.Direction = normalize(mul(camera.rotation, float4(cx, cy,
     ↪  camera.focalLength, 0.0)).xyz);
    ray.TMin = 0.0;
    ray.TMax = 1e4;
    // ...
}
```

Code 29: Camera - Camera Perspective Implementation

The `cx` and `cy` variables are again the coordinates of the current pixel on the ”virtual viewport”. But this time multiplied by `camera.meterPerPixel`, the camera should now correctly render objects measured in meters.
Unlike before, all rays now originate at the camera origin. The direction is just the plane vector from before, offset forwards by the `camera.focalLength`. The last step is to multiply this direction by the rotation matrix and then normalize. We don’t want to apply any translation to the vector, so set the fourth vector component to `0.0`.

The rendered image should now look like this:

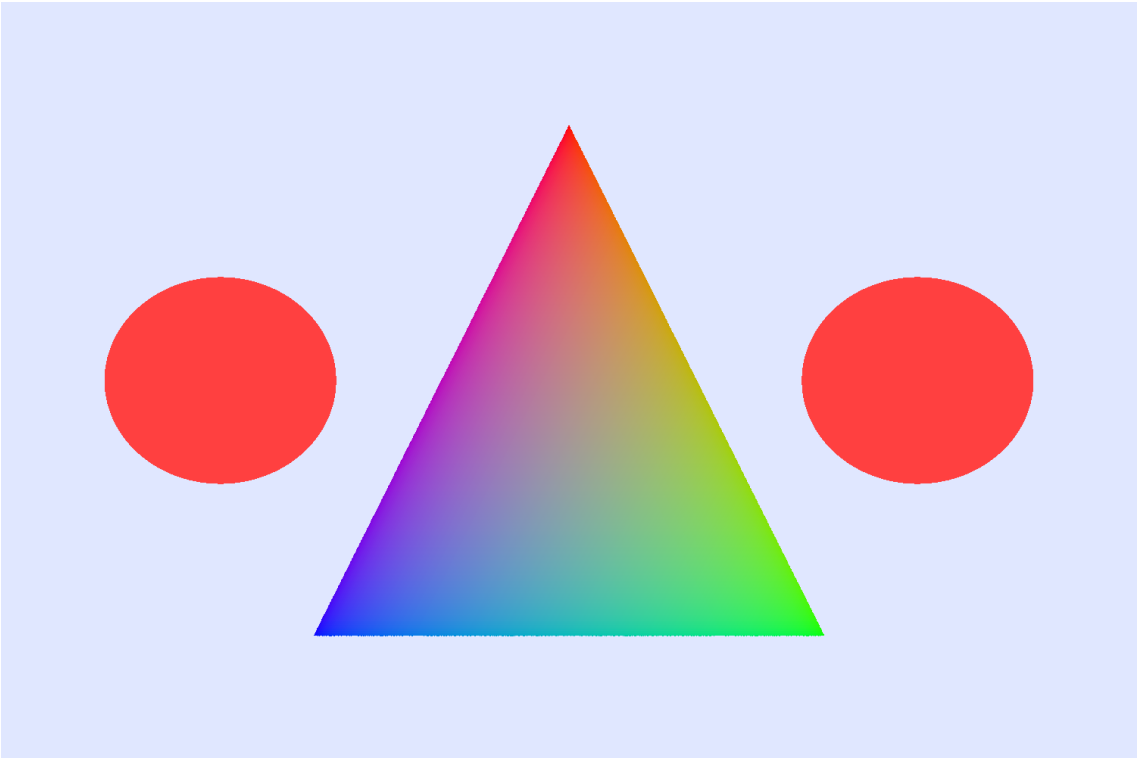

Figure A.4: Camera - Final Result

The spheres now look slightly distorted, as expected with perspective projection. Moving the camera away further from the scene now makes the objects appear smaller.

A.4 Diffuse Lighting

```
- raytracer/src/tutorial/04_diffuse_lighting.cpp
- shaders/tutorial/04_diffuse_lighting.slang
```

There are many different approaches to lighting such Phong shading, which calculates lighting based on the angle between the surface normal and incident light [14]. This computation is very computationally light, it's especially useful for slower CPU based algorithms and real-time. Since this ray tracer runs on very fast specialized hardware, we are able to utilize much more expensive - and better looking - lighting calculations. So this is where we pivot from ray tracing to path tracing [12].

This part of the tutorial was inspired by the popular ray tracing tutorial Ray Tracing In One Weekend. While the idea and approach are similar, the actual implementation differs quite heavily from the *Ray Tracing in One Weekend* tutorial.

Let's start by improving the scene by adding a floor and placing a couple more spheres around the scene. Change the vertices and indices as follows for a basic floor:

```
// ...

const std::vector<Vertex> vertices = {
    {{ -3.0, 0.0, -2.0, 0.0 }},
    {{  3.0, 0.0, -2.0, 0.0 }},
    {{  3.0, 0.0,  2.0, 0.0 }},
    {{ -3.0, 0.0,  2.0, 0.0 }}
};
const std::vector<uint32_t> indices = { 0, 1, 2, 2, 3, 0 };

// ...
```

Code 30: Diffuse Lighting - Floor

Instead of two spheres, let's add five. Additionally also extend the struct with a color (`glm::vec4`).

```
// ...

std::array<Sphere, 5> spheres = {
    Sphere{ {  1.6, 0.45,  1.4 }, 0.45, { 0.20, 0.36, 0.40, 1.0 } },
    Sphere{ {  0.6, 0.30,  0.0 }, 0.30, { 1.00, 0.95, 0.69, 1.0 } },
    Sphere{ { -1.2, 0.20,  0.6 }, 0.20, { 0.88, 0.62, 0.24, 1.0 } },
    Sphere{ { -0.2, 0.25, -0.4 }, 0.25, { 0.62, 0.17, 0.17, 1.0 } },
    Sphere{ { -0.9, 0.10, -0.5 }, 0.10, { 0.33, 0.04, 0.06, 1.0 } },
};

// ...
```

Code 31: Diffuse Lighting - Five Spheres

Right now the camera is on the same height as the floor, so move the camera up a bit: `.origin = glm::vec4(0.0, 0.7, -2.0, 0.0),`.
In the shader, instead of setting all sphere colors to a red, use the color from the `spheres` structured buffer: `payload.color = spheres[InstanceID()].color.xyz;`.
Since the `spheres` buffer is now also accessed inside the *closestHit* shader, set the correct flag for the shader resource on the C++ side using the OR-operator.
Let's also set the color of the floor inside the shader (a more sophisticated color system for triangle structures will be created in a future chapter). In this tutorial the color `float3(0.2, 0.2, 0.3)`.

After these changes, this is the new scene:

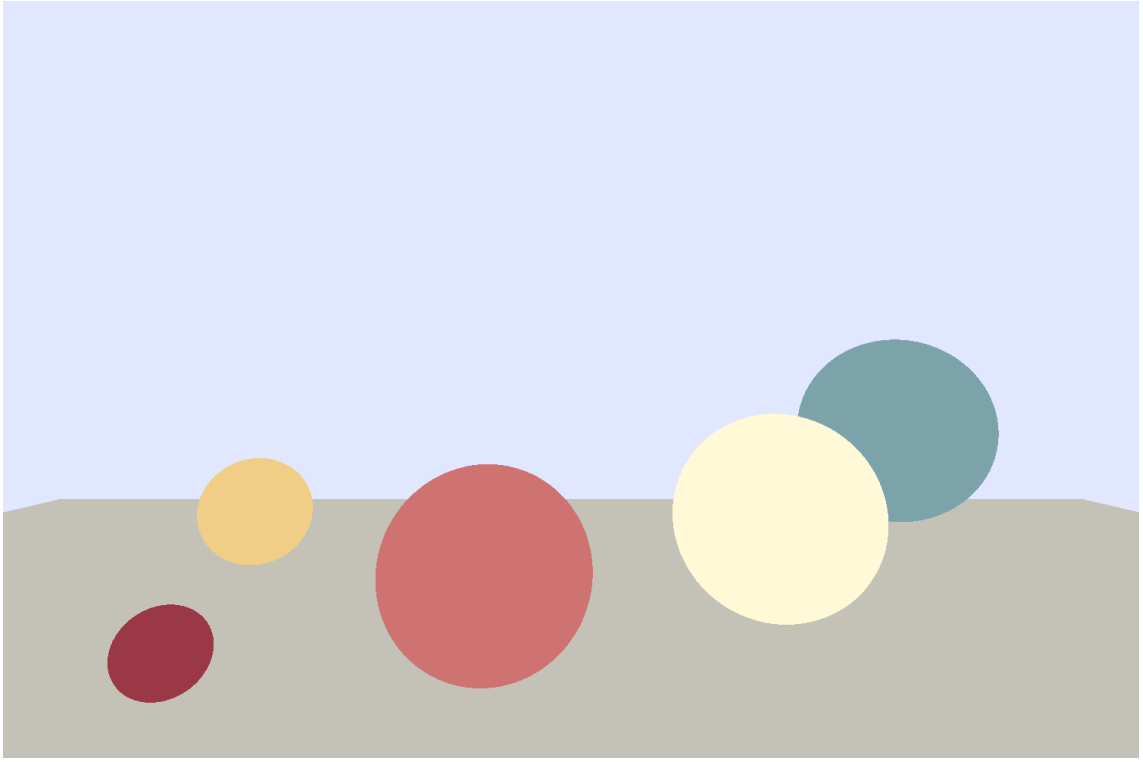

Figure A.5: Diffuse Lighting - New Scene

A.4.1 Lighting Calculation

Onto the actual diffuse lighting calculation. In simple terms, when any geometry is hit, recursively shoot another ray in a random direction into the scene. Additionally, multiply together all encountered colors to get the final pixel color. If a ray hits a light source (in this chapter the background will act as a light source), terminate the process. Repeat this process each frame until the image converges. What this exactly means will become more clear as this chapter goes on.
A more detailed explanation on how this approach works can be found in the aforementioned Ray Tracing In One Weekend tutorial.

Generating a random number on the CPU is quite common and there are numerous options. For a CPU ray tracer, one would typically use a fast, non-cryptographically secure RNG to generate floating point numbers in the range *[0,1)*. On the GPU, a common way to get a "random number" is a simple hashing function. In this tutorial, the functions

defined in this ShaderToy example are used. These offer good performance by not using expensive functions like `sin` and still generate uniformly distributed results.

The hashing function needs an input that satisfies at least these two conditions for optimal results:

1. Different input per pixel, otherwise ugly patterns will show up.
   Use the `launchId` as an input.
2. Different input each frame, otherwise each frame will product the same result.
   Send the current frame index from the GPU to the GPU each frame.
3. Different input for each hit point so recursive rays produce different hashes.
   Use the hit position.

To accumulate pixel colors, another texture that supports larger values is used. Let's add these two new required resources (frame index and accumulation texture) to the *main.cpp* file:

```
// ...

auto accumulationTexture = device.texture(engine::TextureInfo{
    .dimensions = dimensions,
    .format = engine::Format::eR32G32B32A32Sfloat
});
engine::PushConstant<uint32_t> frameConstant;

engine::ShaderResources shaderResources;
shaderResources
    // ...
    .expose(frameConstant, engine::ShaderStageFlags::ClosestHit)
    .expose(accumulationTexture, engine::ShaderStageFlags::Raygen)
    .expose(outputTexture, engine::ShaderStageFlags::Raygen);

// ...
```

Code 32: Diffuse Lighting - Accumulation Texture and Frame Constant

Use the `device.texture` method again to create a new texture, but this time change the format to use 32-bit floating point numbers (the default format is RGBA with an 8-bit integer component).
The second resource is new. It's called a `engine::PushConstant`. This is the most performant way to send small data (less than 128 bytes) to the GPU.

Now we need to update the `frameConstant` each frame:

```
// ...

uint32_t frameIndex = 1;

while (!window.shouldClose()) {
    window.pollEvents();
    renderer.pushConstant(frameConstant, &frameIndex);
    renderer.render(device, outputTexture);
    frameIndex++;
}

// ...
```

Code 33: Diffuse Lighting - Update Frame Constant

Unlike the constant buffer we used for camera settings, push constants are directly pushed to the shader via the renderer (at almost zero cost). But it comes with a disadvantage: only one push constant can be sent to the same shader stage. If you have more than one value, pack the data into a struct and send that to the shader. If you want camera movement, you could send the frequently changing camera rotation matrix using a push constant instead of a constant buffer.

Let's include these resources inside the shader:

```
// ...

[vk::push_constant] const uint frame;

RWTexture2D<float4> accumulationTexture;
RWTexture2D<float4> outImage;

// ...
```

Code 34: Diffuse Lighting - New Shader Resources

Push constants are only supported when using the Vulkan backend. Since Slang supports more than one backend, the variable needs to be marked using the attribute `vk::push_constant`.

Next are two shader functions, that scatter a ray in a random direction:

```
// ...

struct HitPayload {
    float3 color;
    float2 launchId;
    Optional<RayDesc> ray;
}

struct HitRecord {
    float3 position;
    float3 normal;
}

// Hash functions from: https://www.shadertoy.com/view/4djSRW
float3 signedHash32(float2 p) {
    float3 p3 = fract(float3(p.xyx) * float3(.1031, .1030, .0973));
    p3 += dot(p3, p3.yxz+33.33);
    return fract((p3.xxy+p3.yzz)*p3.zyx) * 2.0 - 1.0;
}

float3 diffuseScatter(HitPayload payload, HitRecord record) {
    return normalize(record.normal + normalize(signedHash32(payload.launchId
    ↪  + record.position.zx + float(frame))));
}

// ...
```

Code 35: Diffuse Lighting - Ray Scatter

The first function is a slightly adjusted function from ShaderToy, this version returns values from [-1,1) instead of [0,1). The second function defines a true lambertian reflection, as defined here (Chapter 9.4, True Lambertian Reflection).
To access the `launchId`, pass it via the `HitPayload` struct from the *raygen* to the *closestHit* stage. The third member of the `HitPayload` struct will be used to return a recursive ray from the *closestHit* stage back to the *raygen*.

We can now scatter the recursive rays from the triangle *closestHit* shader entry point as follows:

```
[shader("closesthit")]
void closestHitMain(inout HitPayload payload) {
    HitRecord record;
    record.position = WorldRayOrigin() + RayTCurrent() *
     ↪  WorldRayDirection();
    record.normal = float3(0.0, 1.0, 0.0);

    RayDesc ray;
    ray.Origin = record.position;
    ray.Direction = diffuseScatter(payload, record);
    ray.TMin = 0.001f;
    ray.TMax = 1e4;

    payload.color = float3(0.55, 0.54, 0.47);
    payload.ray = ray;
}
```

Code 36: Diffuse Lighting - Triangle Ray Scatter

To prevent self-intersection, set the `ray.TMin` to something very small. The triangle surface is set to a constant of `float3(0.0, 1.0, 0.0)` (upwards direction), in future chapters these normals will be set on the CPU.

The *closestHit* function for spheres looks similar:

```
[shader("closesthit")]
void sphereClosest(inout HitPayload payload) {
    const Sphere sphere = spheres[InstanceID()];

    HitRecord record;
    record.position = WorldRayOrigin() + RayTCurrent() *
     ↪  WorldRayDirection();
    record.normal = normalize(record.position - sphere.center);

    RayDesc ray;
    ray.Origin = record.position;
    ray.Direction = diffuseScatter(payload, record);
    ray.TMin = 0.001f;
    ray.TMax = 1e4;

    payload.color = sphere.color.xyz;
    payload.ray = ray;
}
```

Code 37: Diffuse Lighting - Sphere Ray Scatter

The normal is calculated by subtracting the sphere center from the hit position. The ray color is still with the color sent from the CPU.

To terminate the ray tracing process, also set the recursive ray inside the `HitPayload` function to `none`:

```
[shader("miss")]
void missMain(inout HitPayload payload) {
    payload.color = float3(0.75, 0.8, 1.0);
    payload.ray = none;
}
```

Code 38: Diffuse Lighting - Terminate Ray Tracing Inside Miss

The last thing to do is supporting recursive rays and updating the accumulation texture. This code block is quite long but most of the code we already looked at previously:

```
static const uint MAX_DEPTH = 6;

[shader("raygeneration")]
void raygenMain() {
    // ...

    HitPayload payload;
    payload.color = float3(0.0);
    payload.launchId = float2(launchId);
    payload.ray = ray;

    float3 color = float3(1.0);

    for (uint i = MAX_DEPTH; i >= 0; --i) {
        if (i == 0) {
            color = float3(0.0);
            break;
        }

        TraceRay(accelerationStructure, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
        ↪ payload.ray.value, payload);
        color *= payload.color;

        if (!payload.ray.hasValue) {
            break;
        }
    }

    accumulationTexture[launchId] += float4(color, 1.0);
    outImage[launchId] = float4(accumulationTexture[launchId].rgb /
    ↪ float(frame), 1.0);
}
```

Code 39: Diffuse Lighting - Sampling and Recursive Rays

There are no recursive calls inside the functions, but it could be implemented recursively.

In this case you would need to set the optional last parameter `maxRecursionDepth` of the `Renderer` constructor. It is generally not recommended to use actual recursion because it's not very widely supported and can have a rather large performance impact.
Once the max recursion depth (in this case 6) is hit or in other words, no light source was reached within six steps, set the color to black. This is what makes shadows visible.
Set the output image color to the average of the accumulated image, by dividing with the current frame index. Also make sure you don't divide by zero here, that's why in the code block Diffuse Lighting - Update Frame Constant the frame index is one based.

After all that work, we finally get a beautiful image with almost realistic lighting:

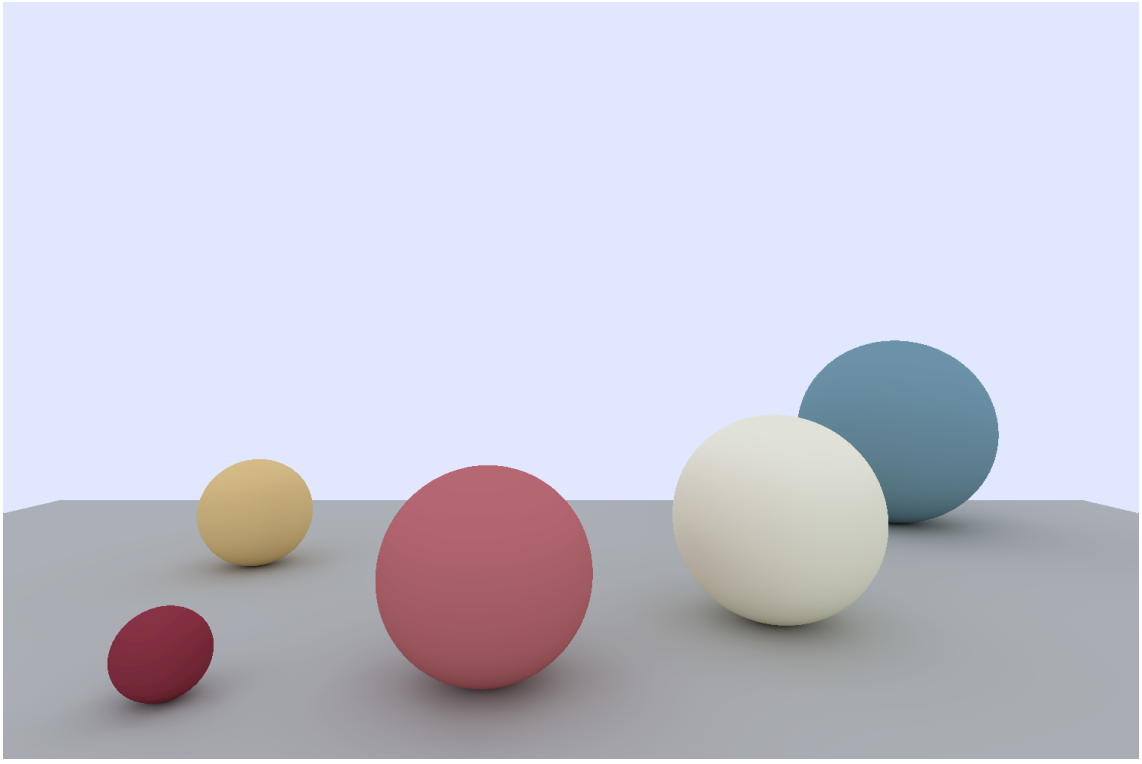

Figure A.6: Diffuse lighting - Final Result

Since this approach is now probabilistic, the image starts off quite noisy but then converges into a smooth image after a short time. Depending on the hardware, setting the `vsync` value to `false` in the *main.cpp* file can massively decrease the time it takes for the image to converge. On modern ray tracing capable hardware, you can expect this scene to run at over 4'000 frames per second. On high-end hardware potentially even more than that.

A.5 Anti-Aliasing

```
- raytracer/src/tutorial/05_anti_aliasing.cpp
- shaders/tutorial/05_anti_aliasing.slang
```

Another improvement that can now easily be implemented is anti-aliasing. A common approach in computer graphics is called multisample anti-aliasing (MSAA) [23]. It involves tracing multiple rays per pixel in a grid like pattern. Instead of a grid pattern, this tutorial randomly offsets initial rays. This approach works better in already probabilistic ray tracers (path tracers), like the one developed here. Only a few minor changes inside the shader are required for this visual improvement:

```
// Hash functions from: https://www.shadertoy.com/view/4djSRW
float hash12(float2 p) {
    float3 p3  = fract(float3(p.xyx) * .1031);
    p3 += dot(p3, p3.yzx + 33.33);
    return fract((p3.x + p3.y) * p3.z);
}

[shader("raygeneration")]
void raygenMain() {
    const float cx = (float(launchId.x) + (hash12(launchId.xy + frame) - 0.5) - float(launchSize.x) * 0.5) * camera.meterPerPixel;
    const float cy = (float(launchSize.y) * 0.5 - float(launchId.y) + (hash12(launchId.yx + frame) - 0.5)) * camera.meterPerPixel;
}
```

Code 40: Anti-aliasing - Implementation

First add a new hash function from ShaderToy that returns a single float. Inside the `raygenMain` function calculate an offset to the current pixel in the range [-0.5,0.5) and add that to the initial ray x and y coordinate.

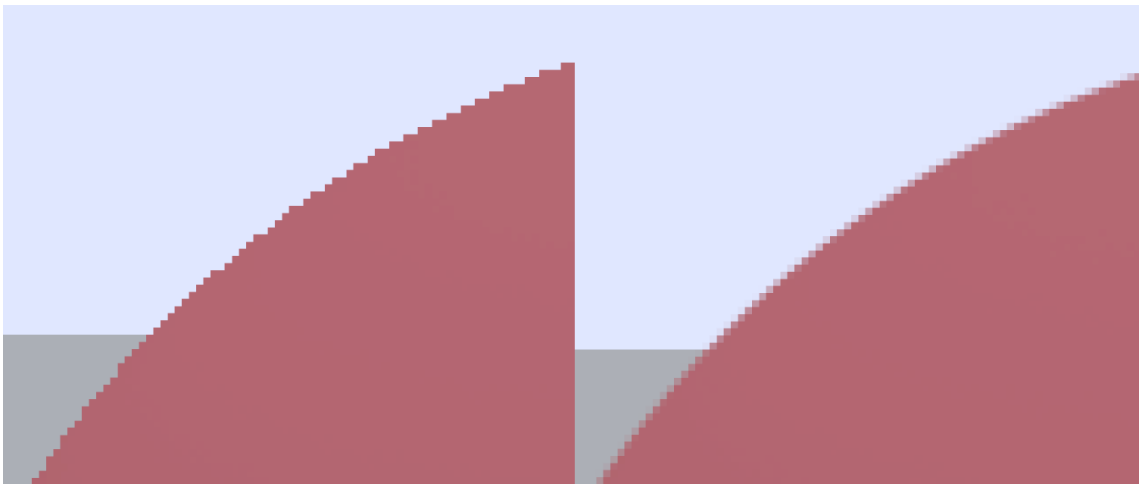

Figure A.7: Anti-Aliasing - Final Result Comparison (left old, right new)

A.6 Reflection

```
- raytracer/src/tutorial/06_reflection.cpp
- shaders/tutorial/06_reflection.slang
```

In this chapter we'll add reflective materials. For brevity, this is only implemented for spheres.
The most obvious approach is to add an enum to the `Sphere` struct specifying the material type. Inside the *closestHit* function you can then call the appropriate scatter function based on the material type. For this tutorial, this approach would be just fine. But if the lighting calculations were more expensive, this could lead to a performance impact due to branching. GPUs heavily rely on SIMD, branching (especially branches with lots of code) breaks this paradigm which can lead to said performance impact [24]. This tutorial uses different *closestHit* shaders for different material types. This removes branching and allows GPUs to schedule rays more efficiently.

Let's start on the C++ side:

```cpp
struct Sphere {
    glm::vec3 center;
    float radius;
    glm::vec3 color;
    uint32_t hitGroupIndex;
};

int main() {
    // ...

    std::array<Sphere, 5> spheres = {
        Sphere{ {  1.6, 0.45,  1.4 }, 0.45, { 0.20, 0.36, 0.40 }, 1 },
        Sphere{ {  0.6, 0.30,  0.0 }, 0.30, { 1.00, 0.95, 0.69 }, 1 },
        Sphere{ { -1.2, 0.20,  0.6 }, 0.20, { 0.88, 0.62, 0.24 }, 2 },
        Sphere{ { -0.2, 0.25, -0.4 }, 0.25, { 0.62, 0.17, 0.17 }, 1 },
        Sphere{ { -0.9, 0.10, -0.5 }, 0.10, { 0.33, 0.04, 0.06 }, 1 },
    };

    for (uint32_t i = 0; i < spheres.size(); ++i)
        asBuilder.add(spheres[i].aabb(), { .instanceId = i, .transform = spheres[i].mat(), .hitGroupIndex = spheres[i].hitGroupIndex });

    engine::ShaderStage intersect(shader, "sphereIntersect");
    shaderStages
        .addGroup(engine::ProceduralHitGroup(intersect, engine::ShaderStage(shader, "sphereClosest")))
        .addGroup(engine::ProceduralHitGroup(intersect, engine::ShaderStage(shader, "reflectiveSphereClosest"))) // NEW

    // ...
}
```

Code 41: Reflection - Dynamic Hit Group Index

Previously, the hit group index was set statically for all spheres, now it is defined per sphere. So create a new `Sphere` member called `hitGroupIndex` and set it inside the array. In this case, the third sphere should be reflective, the others should remain diffuse. Keep in mind that structs sent to the GPU are aligned to 16 bytes. Since the alpha component of the sphere color is not used, change the `glm::vec4` to a `glm::vec3` so the 4-byte `hitGroupIndex` fits inside the struct.
Also add a new group to the `shaderStages`, the entry point will be called `"reflectiveSphereClosest"`.

Let's add the reflective shader function to the *main.slang* file:

```
// ...

struct Sphere {
    float3 center;
    float radius;
    float3 color;
    uint hitGroupIndex;
}

[shader("closesthit")]
void reflectiveSphereClosest(inout HitPayload payload) {
    const Sphere sphere = spheres[InstanceID()];
    const float3 pos = WorldRayOrigin() + RayTCurrent() *
     ↪  WorldRayDirection();
    const float3 normal = normalize(pos - sphere.center);

    RayDesc ray;
    ray.Origin = pos;
    ray.Direction = normalize(reflect(WorldRayDirection(), normal));
    ray.TMin = 0.001f;
    ray.TMax = 1e4;

    payload.color = sphere.color;
    payload.ray = ray;
}

// ...
```

Code 42: Reflection - New Reflective Shader Entry Point

Remember to also add the `hitGroupIndex` to the struct on the shader side, even though it will not be used. The actual reflective shader entry point is very close to the diffuse. Instead of the custom `diffuseScatter` method, we can use the built-in function `reflect`.

And that's already it, reflections can be seen on the back left sphere:

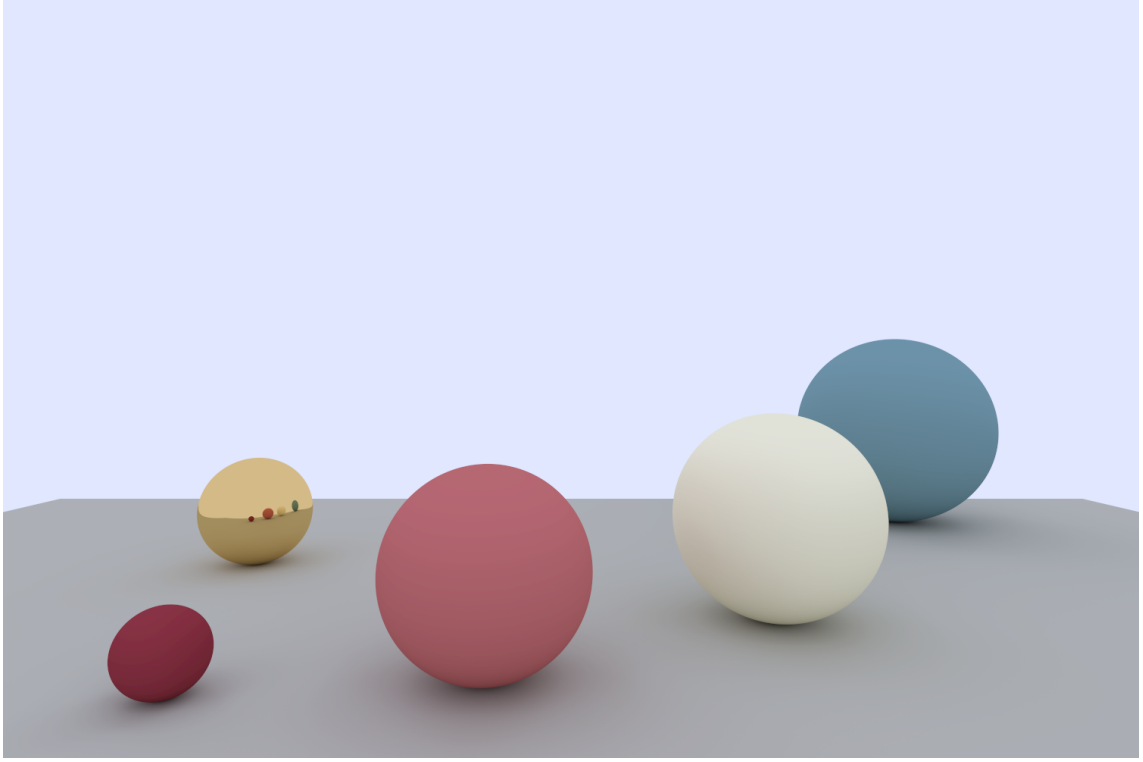

Figure A.8: Reflection - Final Result

Any number of other materials (such as refraction and emission) can be implemented the exact same way. Feel free to implement more materials individually.

A.7 Passing Vertex Data

Right now, the triangle normals are hard-coded in our shader. In this chapter, we're going to send this data to the shader and access it there. Start by extending the `Vertex` struct with the normals and exposing the vertex/index buffers to the shader:

```
struct Vertex {
    glm::vec4 position; // Position MUST always be the first member.
    glm::vec4 normal;
};
const glm::vec4 UP{ 0.0, 1.0, 0.0, 0.0 };
const std::vector<Vertex> vertices = {
    {{ -3.0, 0.0, -2.0, 0.0 }, UP},
    {{  3.0, 0.0, -2.0, 0.0 }, UP},
    {{  3.0, 0.0,  2.0, 0.0 }, UP},
    {{ -3.0, 0.0,  2.0, 0.0 }, UP}
};
const std::vector<uint32_t> indices = { 0, 1, 2, 2, 3, 0 };

int main() {
    // ...

    shaderResources
        // ...
        .expose(vertexBuffer, engine::ShaderStageFlags::ClosestHit)
        .expose(indexBuffer, engine::ShaderStageFlags::ClosestHit)

    // ...
}
```

Code 43: Passing Vertex Data - Extend Vertex Struct with Normals

The acceleration structure only needs position data, but we can also add new data to a `Vertex` that the acceleration structure build process ignores. But this data **MUST** always be the first member of the struct, this is where the build process expects the data.

Let's continue by expanding the shader with a a `Vertex` struct and declare the two new buffers as resources:

```
// ...

struct Vertex {
    float3 position;
    float3 normal;
}
StructuredBuffer<Vertex> vertices[];
StructuredBuffer<uint> indices[];

// ...
```

Code 44: Passing Vertex Data - Add Vertex/Index Resource to Shader

This reveals another feature of the framework, an array of resources denoted with braces before the semicolon. This allows indexing into an array of descriptors and then indexing again to get a specific `Vertex`. This is useful for support multiple meshes. Each mesh has its own vertex/index buffer, we can then easily get the correct buffer in the shader where the vertex/index data of the current mesh is available. Instead of exposing a single `StructuredBuffer` in the *main.cpp* file, you would expose an array/vector of `StructuredBuffers`, the rest is automatically handled by the framework. This is also used in the path tracer example provided with the template project.

```
HitRecord unpackTriangle(float2 bary) {
    const uint instanceId = InstanceID();
    const uint primitiveIndex = PrimitiveIndex();

    const var indexBuffer = indices[NonUniformResourceIndex(instanceId)];
    const var vertexBuffer = vertices[NonUniformResourceIndex(instanceId)];

    const uint i0 = indexBuffer[primitiveIndex * 3 + 0];
    const uint i1 = indexBuffer[primitiveIndex * 3 + 1];
    const uint i2 = indexBuffer[primitiveIndex * 3 + 2];

    const float3 n0 = vertexBuffer[i0].normal;
    const float3 n1 = vertexBuffer[i1].normal;
    const float3 n2 = vertexBuffer[i2].normal;

    const float3 barCords = float3(1.0 - bary.x - bary.y, bary.x, bary.y);
    const float3 position = WorldRayOrigin() + RayTCurrent() *
     ↪ WorldRayDirection();
    const float3 normal = normalize(n0 * barCords.x + n1 * barCords.y + n2 *
     ↪ barCords.z);

    HitRecord record;
    record.position = position;
    record.normal = normal;
    return record;
}
```

Code 45: Passing Vertex Data - Unpacking a Triangle

This method looks intimidating, but is actually quite simple. First get the `instanceId` (make sure to actually increment this on the C++ side when adding multiple meshes) and the `primitiveIndex` (the index of the intersected triangle). Then index into the descriptors to get the buffer of the current mesh. `NonUniformResourceIndex` may not be required on all platforms, but you should add it anyway. If not present, the driver may optimize the index for the current work group, which can lead to artifacts.
Once you have the current vertex/index buffers, use the `primitiveIndex` to get the three indices of the current triangle. These indices are then used to find the three associated vertices. Combine the three normals into the final normal using the barycentric coordinates.

We can now use this function in the *closestHit* shader:

```
[shader("closesthit")]
void closestHitMain(inout HitPayload payload, in
↪ BuiltInTriangleIntersectionAttributes attr) {
    const HitRecord record = unpackTriangle(attr.barycentrics);

    RayDesc ray;
    ray.Origin = record.position;
    ray.Direction = diffuseScatter(payload, record);
    ray.TMin = 0.001f;
    ray.TMax = 1e4;

    payload.color = float3(0.55, 0.54, 0.47);
    payload.ray = ray;
}
```

Code 46: Passing Vertex Data - ClosestHit Unpack Triangles

And that's it, we now fully support multiple large triangle meshes with per-vertex normals. The output image should still look the same.

A.8 Next Steps

Below is a list of possible feature that can be implemented with this framework. You can also consult the *examples* folder for more complicated ray tracers.

- Import triangles from a file like OBJ or GLTF (see *examples* folder)
- Add other procedural shapes
- Add more materials (refraction, emission, ...)
- Make camera interactive
- Read image files and add uv-coordinates to the vertex data (texture mapping)
- Add other effects like motion blur or depth of field